🚀 Collections Framework Masterclass

Chapter 19 — IdentityHashMap — Complete Deep Dive

> **Interview Importance:** ⭐⭐⭐⭐☆

`IdentityHashMap` is a specialized `Map` implementation that intentionally breaks away from normal Java Map equality semantics.

The key idea:

> **`HashMap` compares keys using `equals()`, while `IdentityHashMap` compares keys using reference identity (`==`).**

This makes `IdentityHashMap` useful when you care about:

> **"Are these the exact same object?"**

rather than:

> **"Are these two objects logically equal?"**

---

1. What Is IdentityHashMap?

Package:

```java id="0f4x9s"
java.util.IdentityHashMap
```

```
Example:

```java id="w8q2m6"
Map<String, Integer> map =
    new IdentityHashMap<>();
```
```

The defining behavior is:

```text id="j7v3k1"
IdentityHashMap ↓
key comparison ↓
==
```

```
Whereas:

```text id="x9m5p2"
HashMap
   ↓
key comparison
   ↓
equals()
```

---

2. `==` vs `equals()`

This is the foundation of the entire chapter.

Consider:

```java id="r4k8c3"
String a = new String("Java");
String b = new String("Java");
```

Now:

```java id="e6p2m7"
System.out.println(a.equals(b));
```

Output:

```text id="z3q9v1"
true
```

Because both contain the same characters.

But:

```java id="n7w5k2"
System.out.println(a == b);
```

Output:

```text id="m2x8c4"
false
```

Because they are different objects.
```

```
---

# 3. HashMap Behavior

With:

```java id="a8f3q6"
Map<String, Integer> map =
    new HashMap<>();

map.put(a, 100);
map.put(b, 200);
```

Because:

```text id="y5p1n8" a.equals(b) == true
```

```
the second insertion replaces the first mapping.

Final:

```text id="c4m7x2"
one logical key → 200
```

---

4. IdentityHashMap Behavior

Now:

```java id="q6v9k3" Map<String, Integer> map = new IdentityHashMap<>();

map.put(a, 100); map.put(b, 200);
```

```
Here:

```text id="p2r8m5"
a == b
```

is:

```text id="g3x7n1" false
```
```

```
Therefore both keys can coexist:

```text id="h9k4c2"
a → 100
b → 200
```

This is the defining difference.

---

5. Side-by-Side

```text id="q7m3v8" HashMap IdentityHashMap | | equals() == | | logical equality object identity

```
Example:

```text id="x5k8p1"
a.equals(b) → true
a == b      → false
```

Result:

```text id="m4r9c6" HashMap → treats a and b as same key

IdentityHashMap → treats a and b as different keys

```
---

6. Why Would Anyone Want This?

Most business applications care about:

```text id="v8p2m5"
logical equality
```

For example:

```text id="d1k6q9" Employee ID 101

```
Two Employee objects representing ID 101 might logically be the same employee.
```

```
So HashMap is appropriate.

But some algorithms care about:

```text id="z4n7c2"
object identity
```

For example:

> "Have I already encountered this exact object instance?"

That's where IdentityHashMap becomes useful.

---

7. Object Graph Traversal

One important use case is processing object graphs.

Suppose:

```text id="s8m3v5"
Object A → Object B → Object A
```

```
There is a cycle.

If you're traversing the graph and want to detect whether you've already visited
the **same object instance**, identity semantics are useful.

```java id="u5x9q2"
Map<Object, Boolean> visited =
    new IdentityHashMap<>();
```

Then:

```java id="r3k7m1"
if (visited.put(object, Boolean.TRUE) == null) {
    // first time seeing this exact object
}
```

```
The identity of the object matters, not its `equals()` implementation.

---

# 8. Why `HashMap` Can Be Wrong for Identity-Based Algorithms

Suppose two different objects implement:
```

```java id="j8p4v6"
equals()
```

such that:

```text id="c2m9x5" objectA.equals(objectB) == true
```

```
A HashMap would treat them as the same key.

But your algorithm might need:

```text id="f7q3n1"
objectA != objectB
```

to mean:

```text id="w6r8k2" different objects
```

```
IdentityHashMap gives you that behavior.

---

# 9. IdentityHashMap Uses `==`

This is the rule to remember:

```text id="m8x2q5"
IdentityHashMap key comparison:
     reference equality
          ↓
         ==
```

Not:

```text id="p4v7n9" equals()
```

```
---

# 10. What About `hashCode()`?

IdentityHashMap also uses identity-based hashing semantics.
```

````
Conceptually, it uses:

```java id="c8r5m2"
System.identityHashCode(object)
````

rather than relying on the object's overridden:

```java id="v1q6k9" object.hashCode()

````
This is important.

Why?

Because the Map wants hashing to correspond to:

```text id="j3m8p4"
object identity
````

rather than logical equality.

---

11. `System.identityHashCode()`

Java provides:

```java id="h7x2n5" System.identityHashCode(object)

````
This returns the identity-based hash associated with the object.

Even if the class overrides:

```java id="a4q9m6"
hashCode()
````

`System.identityHashCode()` provides the identity-based hash semantics.

---

12. Example

Suppose:
````

```java id="k5v8p3" class Employee {

```
int id;

@Override
public int hashCode() {
    return id;
}

@Override
public boolean equals(Object o) {
    ...
}
```

} `Two Employees may have:` text id="s7n3m9" same id ```

and therefore:

```text id="q2x6p8" same equals() same hashCode()

```
But IdentityHashMap still distinguishes them if they are different object
instances.

---

13. Internal Structure

IdentityHashMap does not simply behave like:

```text id="u8m4r2"
HashMap + replace equals() with ==
```

Its internal representation is specialized.

Conceptually, it uses an array-based table structure containing alternating:

```text id="c7p2m9" key value key value key value

```
rather than the normal HashMap bucket-node structure.

This is an important implementation difference.

```

```
# 14. Why an Array Instead of HashMap Nodes?

IdentityHashMap is optimized for its specialized semantics.

Conceptually:

```text id="d5x8q1"
IdentityHashMap table:

[ key ][ value ][ key ][ value ][ key ][ value ]
```

This avoids creating separate Node objects for each mapping.

The implementation uses open-addressing-style probing.

---

15. Open Addressing

Unlike HashMap's:

```text id="v9m3x6" bucket ↓ Node → Node → Node

```
IdentityHashMap uses an array and searches for a suitable slot when collisions
occur.

Conceptually:

```text id="r2k7p4"
hash
 ↓
initial slot
 ↓
occupied?
 ├── no  → use it
 └── yes → probe another slot
```

This is called:

> **Open addressing**

```

16. IdentityHashMap vs HashMap Internals

HashMap

```text id="x4p9m2" array of buckets ↓ Node ↓ Node → Node

```
Potentially tree bins for heavy collisions.

IdentityHashMap

```text id="f8k3q6"
single array
 ↓
key/value pairs
 ↓
open addressing
```

This is a major implementation difference.

---

17. Collision Handling

In HashMap:

```text id="h3m7v1" collision ↓ same bucket ↓ linked nodes / tree bin

```
In IdentityHashMap:

```text id="p5x8c2"
collision
 ↓
probe another array slot
```

So it doesn't use HashMap-style linked bucket chains.

---

18. Why IdentityHashMap Doesn't Use `equals()`

Imagine:

```java id="w7n2k4" Object a = new Object(); Object b = new Object();
```

````
Even if a custom class says:

```text id="m8c3p6"
a.equals(b) == true
````

IdentityHashMap wants:

```text id="q4x9r1" a == b

````
The semantic contract is different.

This makes it appropriate for identity-sensitive operations.

---

19. Important Map Contract Difference

The Java `Map` contract is normally based on logical equality.

IdentityHashMap intentionally uses **reference equality** instead.

The JDK documentation explicitly describes this as a deliberate departure from
the general `Map` contract.

This is why IdentityHashMap is a specialized collection rather than a normal
replacement for HashMap.

---

20. `containsKey()`

IdentityHashMap's:

```java id="r8m3k5"
containsKey(key)
````

checks identity.

So if:

```java id="v2x7q9" a != b

````
even when:

```text id="s5k1m8"
a.equals(b) == true
````

then:

```java id="t4p6n2"
map.containsKey(a)
```

````
and:

```java id="y8c3r5"
map.containsKey(b)
````

can produce different results depending on which exact object was inserted.

---

21. `remove()`

Likewise:

```java id="j6m9q2"
map.remove(a);
```

````
removes the mapping associated with the exact object reference `a`.

A distinct but equal object `b` won't remove it.

---

# 22. Null Keys

IdentityHashMap allows null keys.

```java id="n4x7p1"
map.put(null, "Hello");
````

There can be one null key mapping, just like Map key uniqueness generally requires.

````

23. Null Values

Null values are also permitted.

```java id="q9m2v6" map.put(object, null);

```
So:

```text id="a5k8r3"
null key   → allowed
null value → allowed
```

---

24. Duplicate Keys?

IdentityHashMap doesn't allow duplicate **identical references**.

But it can contain multiple distinct objects that are equal according to `equals()`.

Example:

```text id="c7v4m9" a.equals(b) → true a == b → false

```
Both can exist as keys.

---

25. Duplicate Values

Values can be duplicated.

```java id="z3p6x8"
map.put(a, "Java");
map.put(b, "Java");
```

Valid.

---

26. IdentityHashMap and Mutable Objects

IdentityHashMap behaves differently from HashMap regarding mutation of an object's overridden `hashCode()`.

Suppose:

```java id="y8m3r5" class Employee { int id;

```
@Override
public int hashCode() {
    return id;
}
```

} `Changing:` java id="p6q2v9" employee.id ```

doesn't change the identity hash used by IdentityHashMap.

So the classic HashMap problem involving a mutable logical hash is not the same here.

However:

> You still shouldn't casually mutate objects involved in complex identity-based algorithms, because object state can affect the algorithm's logic even if it doesn't affect identity hashing.

---

# 27. Identity Is Stable

An object's identity remains associated with that object instance during its lifetime.

Therefore:

```text id="f5r8k2" Object instance ↓ same identity

```
even if:

```text id="n3x7m1"
fields change
```
```

This is exactly what identity-based structures need.

---

28. `equals()` vs Identity

Consider:

```java id="b7m4q8"
class Person {

    String name;

    @Override
    public boolean equals(Object o) {
        Person p = (Person) o;
        return name.equals(p.name);
    }

    @Override
    public int hashCode() {
        return name.hashCode();
    }
}
```

Now:

```java id="g2x9m5"
Person p1 = new Person("John");
Person p2 = new Person("John");
```

Then:

```text id="q5r8c3"
p1.equals(p2) → true
p1 == p2 → false
```

HashMap:

```text id="v6n2k9"
p1 and p2 → same logical key
```

IdentityHashMap:

```text id="m4x7p1"
p1 and p2 → different keys
```

---

29. ⭐ Classic Interview Question

What is the difference between HashMap and IdentityHashMap?

Strong answer:

````
> HashMap uses `equals()` and the corresponding `hashCode()` contract for key equality, whereas IdentityHashMap uses reference identity (`==`) and identity-based hashing. Therefore, two distinct objects that are logically equal can coexist as separate keys in IdentityHashMap.

---

# 30. ⭐ Classic Interview Question

### Why would you use IdentityHashMap?

Strong answer:

> When object identity matters more than logical equality—for example, tracking visited object instances during object-graph traversal, preserving object identity during graph copying, or associating metadata with exact object instances.

---

# 31. ⭐ Classic Interview Question

### What is `System.identityHashCode()`?

Answer:

> It returns an identity-based hash value for an object, independent of the object's overridden `hashCode()` implementation.

---

# 32. ⭐ Classic Interview Question

### Can two equal objects exist as separate IdentityHashMap keys?

Yes.

If:

```text id="u3k7m9"
a.equals(b) == true
````

but:

```text id="v8x2p4"
a != b
```
```

```
IdentityHashMap treats them as different keys.

---

# 33. ⭐ Classic Interview Question

### Does IdentityHashMap use `hashCode()`?

Not in the normal logical-equality sense.

It uses identity-based hashing, conceptually associated with:

```java id="h4q9m2"
System.identityHashCode(key)
```

rather than the key's overridden `hashCode()`.

---

34. ⭐ Coding Question

What is the output?

```java id="r8m4x1" String a = new String("Java"); String b = new String("Java");

Map<String, Integer> map = new IdentityHashMap<>();

map.put(a, 10); map.put(b, 20);

System.out.println(map.size());
```

Output:

```text id="j3v7q5"
2
```

Because:

```text id="m5x8k2" a != b
```

```
---

# 35. Same Code with HashMap
```
```

```java id="p4n9c6"
Map<String, Integer> map =
    new HashMap<>();

map.put(a, 10);
map.put(b, 20);

System.out.println(map.size());
```

Assuming `a.equals(b)` is true:

```text id="k7m2x8" 1
```

```
The second value replaces the first.

---

# 36. ⭐ Coding Question

```java id="w3q8m1"
String a = new String("Java");
String b = new String("Java");

IdentityHashMap<String, Integer> map =
    new IdentityHashMap<>();

map.put(a, 10);

System.out.println(
    map.containsKey(b)
);
```

Output:

```text id="r6x2p9" false
```

```
Because `b` isn't the same object as `a`.

---

# 37. ⭐ Coding Question

```java id="q4m8v2"
```

```
IdentityHashMap<String, Integer> map =
    new IdentityHashMap<>();

String a = new String("Java");

map.put(a, 10);

System.out.println(
    map.containsKey(a)
);
```

Output:

```text id="f7x3k5" true

```
Same exact reference.

---

38. ⭐ Tricky Question — String Pool

Consider:

```java id="b9m4q7"
String a = "Java";
String b = "Java";
```

What about:

```java id="x2k8n5" a == b

```
It can be:

```text id="c6p3r9"
true
```

because string literals can refer to the same interned String object.

Therefore:

```java id="h5q7m1" IdentityHashMap<String, Integer> map = new IdentityHashMap<>();

map.put(a, 10); map.put(b, 20);
```

```
can result in:

```text id="n8x4v2"
size = 1
```

because `a` and `b` may be the exact same reference.

This is why you must understand `==`, not just `equals()`.

---

39. `new String()` Changes the Situation

```java id="z5m2q8" String a = new String("Java"); String b = new String("Java");
```

```
Now:

```text id="r7x3n6"
a == b → false
```

Therefore IdentityHashMap can store both.

---

40. IdentityHashMap and Serialization

One legitimate use case is supporting algorithms where **object identity must be preserved**.

Suppose an object graph contains:

```text id="p8m4x2" A ├── B └── B
```

```
where both references point to the **same B object**.

An identity map can record:

```text id="y3q7k1"
original B → copied B
```

so that both references can point to the same copied object.

This prevents accidentally creating two separate copies of the same original object.
```

41. Object Graph Copying

Conceptually:

```text id="m6x2p9"
Original graph:

A ├──── B
  └──── B
        ↑ same object
```

```
During copying:

```text id="j4r8v5"
IdentityMap:

Original A → Copy A
Original B → Copy B
```

When B is encountered again:

```text id="s9k3q7"
Already mapped?
        ↓ YES
        ↓ Reuse Copy B
```

```
This preserves object identity relationships.

---

# 42. IdentityHashMap Is Not a General Replacement for HashMap

Don't use:

```java id="w2m7p4"
IdentityHashMap
```

just because it is another Map.

For ordinary business logic:

```text id="v5x9k3"
HashMap
```

```
is almost always the more natural choice.

Use IdentityHashMap only when:
```

```text id="q8m4r1"
object identity
```

is genuinely part of the requirement.

---

43. IdentityHashMap vs WeakHashMap

These solve completely different problems.

IdentityHashMap

```text id="a6p2x8" How are keys compared? ↓ ==

```
WeakHashMap

```text id="r3k7m9"
How are keys retained?
       ↓
weak references
```

So:

```text id="f5q8m2" IdentityHashMap → identity semantics WeakHashMap → weak lifecycle semantics

```
---

44. IdentityHashMap vs HashMap

Feature	HashMap	IdentityHashMap
Key equality	`equals()`	`==`
Hashing	`hashCode()`	Identity-based
Equal distinct objects	Same key	Separate keys
Null key	One	One
Null values	Yes	Yes
Internal structure	Buckets/nodes	Array + probing
Treeification	Yes, for eligible bins	❌
Thread-safe	❌	❌
General-purpose	✅	❌
Identity-sensitive	❌	✅

```
# 45. IdentityHashMap vs TreeMap

TreeMap asks:

```text id="z4m8q1"
How do keys compare?
```

using:

```text id="k6x2p9" Comparator / Comparable
```

```
IdentityHashMap asks:

```text id="r7v3m5"
Are these the same object?
```

using:

```text id="n2q8k4"

```
Completely different semantics.

---

# 46. IdentityHashMap and `entrySet()`

You can still use:

```java id="x6m3p8"
map.entrySet();
```

just like other Maps.

But key equality behavior remains identity-based.

So:

```java id="q2r7k5" entry.getKey()
```

```
is the exact object reference used as the key.
```
```

````
---

# 47. IdentityHashMap and `containsValue()`

The identity semantics aren't limited only to keys in the conceptual API.

The implementation/documentation specifies identity-based comparisons for keys
and values where applicable.

The important interview point is:

> IdentityHashMap is designed around reference identity rather than ordinary
logical equality.

Don't assume it behaves exactly like HashMap with only one tiny method changed.

---

# 48. Iteration Order

IdentityHashMap does **not** provide insertion-order or sorted-order guarantees.

If you need:

```text id="v4n8m2"
insertion order
````

use:

```text id="x7q3p9" LinkedHashMap
````

````
If you need:

```text id="k5m1r8"
sorted keys
````

use:

```text id="z3x6q2" TreeMap
````

````
---

# 49. Capacity
````

````
IdentityHashMap has an expected maximum-size concept in its constructor:

```java id="p9m4v7"
new IdentityHashMap<>(100);
````

The parameter represents an expected maximum number of entries rather than behaving exactly like HashMap's initial bucket capacity.

This is another subtle implementation/API distinction.

---

50. Performance

IdentityHashMap's average basic operations are generally:

```text id="q3x8m1" O(1)

```
because it uses hashing and open addressing.

However:

> Performance depends on hash distribution, table occupancy, and probing.

It should not be selected solely because it uses an array.

The primary reason to select it is **identity semantics**.

---

51. Why Open Addressing?

In an open-addressed table:

```text id="c7m2p9"
slot 0
slot 1
slot 2
slot 3
...
```

If the preferred slot is occupied:

```text id="n8x4r1" try another slot

````
This continues until the appropriate key is found or an empty slot is found.

This differs from HashMap's bucket chaining.

---

52. Conceptual Lookup

For:

```java id="m5q8v3"
map.get(key);
````

think:

```text id="g7x2p4"
identityHashCode(key) ↓
initial slot ↓
same reference? ↙ ↘
YES NO ↓
↓ value probe
next slot ↓
repeat
````

````
The key comparison is:

```java id="j4r9n6"
storedKey == key
````

not:

```java id="p8m3x7"
storedKey.equals(key)
````

````
---

# 53. Why `equals()` Can Be Dangerous Here

Suppose a class has:

```java id="v5k2q8"
equals()
````

based on business identity:

```text id="m8x3p1"
employeeId
````
````

```
Two different object instances may represent the same employee.

HashMap:

```text id="q7r4n9"
same employee
→ same key
```

IdentityHashMap:

```text id="a2m6x8" different object instances → different keys
```

```
That can be exactly what an object-graph algorithm needs.

---

# 54. ⭐ Interview Scenario

### Requirement

You are traversing an object graph and need to detect whether the **exact same
object instance** has already been visited.

Use:

```java id="r6x2m9"
IdentityHashMap<Object, Boolean>
```

---

55. ⭐ Interview Scenario

Requirement

You want:

```text id="j3p7v5" Customer ID → Customer
```

```
where two Customer objects with the same ID should be treated as the same
logical entity.

Use:
```

```text id="m8q2x4"
HashMap
```

not IdentityHashMap.

---

56. ⭐ Interview Scenario

Requirement

You want to preserve insertion order.

Use:

```text id="k5v9m2" LinkedHashMap

```
not IdentityHashMap.

---

57. ⭐ Interview Scenario

Requirement

You need keys sorted numerically.

Use:

```text id="p4x7n1"
TreeMap
```

not IdentityHashMap.

---

58. ⭐ Interview Scenario

Requirement

You want entries to disappear when their keys are no longer strongly reachable.

Use:

```text id="q8m3v6" WeakHashMap

```
not IdentityHashMap.

---

59. ⭐ Tricky Interview Question

Does IdentityHashMap violate the `Map` contract?

The JDK documentation describes IdentityHashMap as intentionally violating the
general `Map` contract's use of `equals()` for key equality because it
deliberately uses reference equality.

This is not an accidental bug.

It is the defining purpose of the class.

---

60. ⭐ Tricky Interview Question

If `a.equals(b)` is true, can both be keys in IdentityHashMap?

Yes.

If:

```text id="w4m8p2"
a != b
```
```

then they are distinct identity keys.

---

# 61. ⭐ Tricky Interview Question

**If `a.hashCode() == b.hashCode()`, can IdentityHashMap still treat them as different keys?**

Yes.

Hash equality does not imply identity.

IdentityHashMap ultimately distinguishes references using:
```

```text id="s3q7n5"

````
---

# 62. ⭐ Tricky Interview Question

### If `a == b`, can IdentityHashMap contain two separate entries for them?

No.

They are the exact same object reference, so they represent the same identity
key.

---

# 63. ⭐ Coding Question

```java id="v9m4q2"
class Person {

    int id;

    Person(int id) {
        this.id = id;
    }

    @Override
    public boolean equals(Object obj) {
        return obj instanceof Person
            && ((Person) obj).id == id;
    }

    @Override
    public int hashCode() {
        return id;
    }
}
````

Then:

```java id="j2x8p5" Person p1 = new Person(10); Person p2 = new Person(10);

IdentityHashMap<Person, String> map = new IdentityHashMap<>();

map.put(p1, "A"); map.put(p2, "B");
```
```

````
What is:

```java id="n6r3m7"
map.size();
````

Answer:

```text id="q4v8x2" 2
````

````
Because:

```text id="f7m1p9"
p1.equals(p2) → true
p1 == p2      → false
````

---

64. ⭐ Same Question with HashMap

```java id="a8q3m6" HashMap<Person, String> map = new HashMap<>();

map.put(p1, "A"); map.put(p2, "B");
````

````
Now:

```text id="z5r7x1"
map.size() → 1
````

assuming the shown `equals()` and `hashCode()` implementation.

---

65. ⭐ Coding Question

```java id="p6m2v9" Person p1 = new Person(10); Person p2 = p1;

IdentityHashMap<Person, String> map = new IdentityHashMap<>();

map.put(p1, "A"); map.put(p2, "B");

````
Result:

```text id="r8x4k3"
size = 1
````

because:

```text id="j7m5q2" p1 == p2

````
The second `put()` replaces the value.

---

66. ⭐ Coding Question – String Literals

```java id="w3p8m1"
String a = "Java";
String b = "Java";

IdentityHashMap<String, Integer> map =
    new IdentityHashMap<>();

map.put(a, 10);
map.put(b, 20);
````

Because string literals are interned, `a` and `b` can refer to the same String object:

```text id="k4q7n2" a == b → true

````
Therefore:

```text id="m9x3v6"
size = 1
````

This is a good interview trap.

---

67. ⭐ Coding Question — `new String()`

```java id="h5m8r2" String a = new String("Java"); String b = new String("Java");

IdentityHashMap<String, Integer> map = new IdentityHashMap<>();

map.put(a, 10); map.put(b, 20);

```
Now:

```text id="q6x1p9"
a == b → false
```

Therefore:

```text id="v3m7k4" size = 2
```

```
---

# 68. IdentityHashMap Decision Framework

```text id="r2k8m5"
Need Map
   ↓
How should keys be compared?
   ↓
 ┌──────────────┬──────────────┬──────────────┐
 ↓              ↓              ↓
equals()    identity ==     ordering
 ↓              ↓              ↓
HashMap   IdentityHashMap  TreeMap
```

Ordering requirement:

```text id="x7m3p9" Insertion/access order ↓ LinkedHashMap
```

```
Lifecycle requirement:

```text id="n4q8k2"
Weak keys
   ↓
WeakHashMap
```

```

69. Complete Map Selection

At this stage:

```text id="s5v9m3"
                    Map
                     |
     ┌───────────────┼──────────────┐
     ↓               ↓              ↓
  HashMap      LinkedHashMap     TreeMap
     ↓               ↓              ↓
Fast lookup   Predictable order   Sorted keys
                     |
                     ↓
                Access order
                     ↓
                    LRU
```

```
Specialized:

```text id="f8m2q6"
WeakHashMap
   ↓
Weak keys

IdentityHashMap
   ↓
Identity keys

EnumMap
   ↓
Enum keys

ConcurrentHashMap
   ↓
Concurrent access
```

🧠 Final Mental Model

The entire IdentityHashMap chapter can be reduced to:

```text id="q3x7m1"
                HashMap
                   |
            logical equality
                   ↓
                equals()
```

```
    IdentityHashMap
            |
      object identity
            ↓
            ==
```

`Example:` text id="m8p4r2" a.equals(b) → true a == b → false ```
```

Then:

```text id="k5v9x3" HashMap → one logical key

IdentityHashMap → two identity keys

```
Internal difference:

```text id="w7m2q8"
HashMap
→ buckets + nodes

IdentityHashMap
→ array + open addressing
```

🔥 Chapter Summary

Concept	Key Point
IdentityHashMap	Identity-based Map
Key comparison	`==`
Normal HashMap	`equals()`
Hashing	Identity-based
Identity hash	`System.identityHashCode()` concept
Internal structure	Array + open addressing
HashMap-style buckets	❌
Treeification	❌
Null key	✅
Null values	✅
Duplicate equal objects	Can coexist
Same object reference	One key
Ordering	None guaranteed
Thread-safe	❌
Main use	Identity-sensitive algorithms

Concept	Key Point
Object graph traversal	Common use case
Graph copying	Useful
General-purpose Map	❌

🎯 Interview Readiness Checklist

You should be able to explain:

- [ ] `==` vs `equals()`
- [ ] Why IdentityHashMap exists
- [ ] HashMap vs IdentityHashMap
- [ ] Identity-based key comparison
- [ ] Identity-based hashing
- [ ] `System.identityHashCode()`
- [ ] How equal-but-distinct objects behave
- [ ] How identical references behave
- [ ] Internal array representation
- [ ] Open addressing
- [ ] Collision handling through probing
- [ ] Why it doesn't use HashMap-style buckets
- [ ] Object graph traversal use case
- [ ] Graph-copying use case
- [ ] Null behavior
- [ ] Why it isn't a general replacement for HashMap
- [ ] IdentityHashMap vs WeakHashMap
- [ ] IdentityHashMap vs TreeMap
- [ ] String-pool interview traps
- [ ] Mutable-object implications
- [ ] Why IdentityHashMap intentionally departs from normal Map equality semantics

---

🚀 Next Chapter

Chapter 20 — EnumMap (Complete Deep Dive)

We'll cover:

- Why EnumMap is different from HashMap
- Internal array-based implementation

- Why EnumMap is extremely efficient
- Enum ordinal and array indexing
- Type safety
- Natural enum declaration order
- Null-key restriction
- Null values
- EnumMap vs HashMap
- EnumMap vs EnumSet
- Practical state-machine examples
- Configuration/state lookup examples
- Performance and memory advantages
- 20+ interview questions